

AWS Application Load Balancer Project

Introduction

This project demonstrates how to configure an AWS Application Load Balancer (ALB) with multiple EC2 instances and target groups.

Different applications are hosted on separate EC2 instances, and the Application Load Balancer routes traffic based on URL paths such as `/mobile` and `/laptop`.

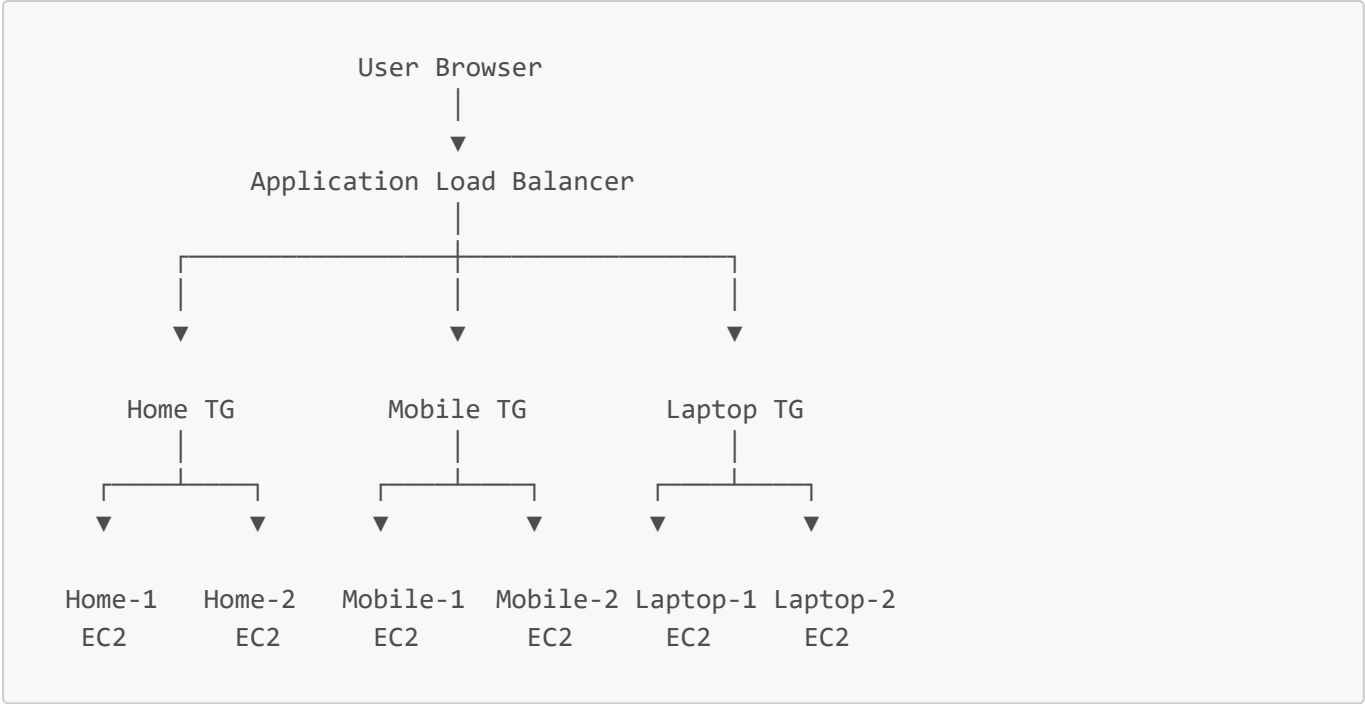
This is an extended architecture project where multiple EC2 servers are attached to each service for better scalability and high availability.

The Application Load Balancer distributes incoming traffic between multiple servers using the Round Robin algorithm.

This project helps in understanding:

- AWS Load Balancer
- Target Groups
- Path-Based Routing
- Round Robin Technique
- EC2 Integration
- High Availability Architecture
- Traffic Distribution

Architecture

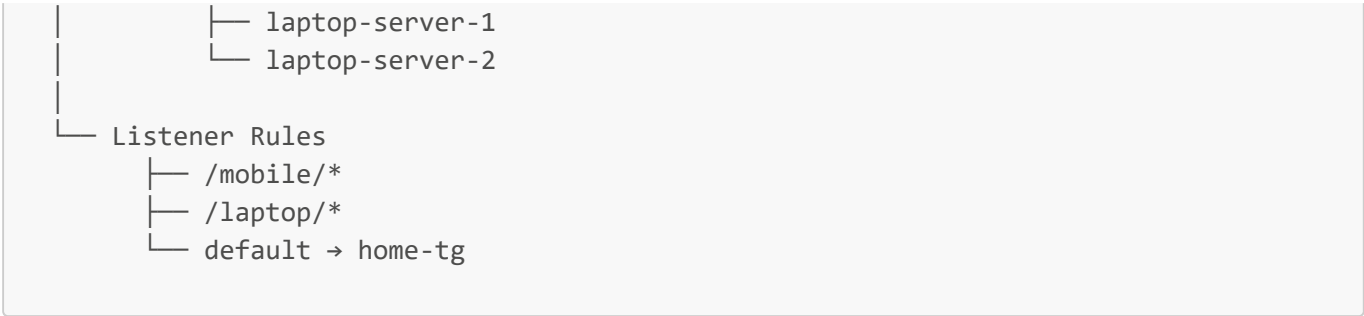


AWS Services Used

Service	Purpose
EC2	Hosting Applications
Application Load Balancer	Traffic Distribution
Target Groups	Route Requests
Security Groups	Allow HTTP Traffic

Project Structure





Step 1 — Launch EC2 Instances

Launch six EC2 instances:

Service	Instances
Home	2 EC2 Servers
Mobile	2 EC2 Servers
Laptop	2 EC2 Servers

Use:

- Amazon Linux 2023
- t3.micro instance type

Allow ports:

- SSH → 22
- HTTP → 80

Step 2 — Create Target Groups

Create three target groups:

Target Group	EC2 Instances
home-tg	home-server-1, home-server-2
mobile-tg	mobile-server-1, mobile-server-2
laptop-tg	laptop-server-1, laptop-server-2

Protocol:

- HTTP

Port:

- 80

Target Type:

- Instance
-

Step 3 — Create Application Load Balancer

Create an Application Load Balancer with:

- Internet Facing
 - HTTP Listener on Port 80
 - Attach all target groups
-

Step 4 — Configure Listener Rules

Add path-based routing rules:

Path	Target Group
/mobile/*	mobile-tg
/laptop/*	laptop-tg
Default	home-tg

Round Robin Technique

The Application Load Balancer uses the Round Robin algorithm to distribute incoming requests equally among multiple EC2 instances inside a target group.

Example:

- First request → home-server-1
- Second request → home-server-2
- Third request → home-server-1
- Fourth request → home-server-2

This technique helps in:

- Load Distribution
 - Better Performance
 - High Availability
 - Reduced Server Load
 - Improved Scalability
-

Application URLs

Home Page

```
http://application-lb-dns-name/
```

Mobile Page

```
http://application-lb-dns-name/mobile/
```

Laptop Page

```
http://application-lb-dns-name/laptop/
```

Screenshots

EC2 Running Instances

Instances (6) Info

Last updated less than a minute ago

Connect

Instance state

Actions

Launch instances

Saved filter sets

Choose filter set

Find Instance by attribute or tag (case-sensitive)

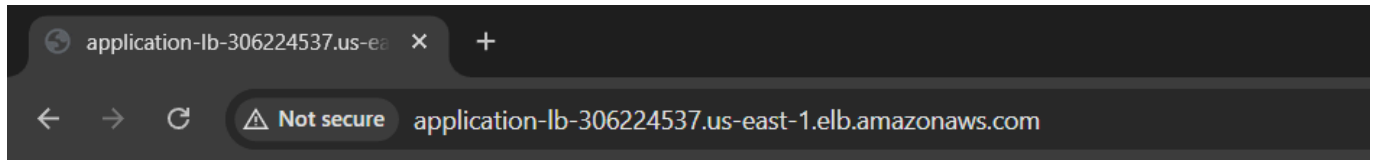
Instance state = running

Clear filters

	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	
☐	home	i-09b45ffa154d8fa0e	Running	t3.micro	3/3 checks passed	us-east-1c	ec2-3-82-23-56.comput...	3
☐	mobile	i-0db036ad3ba830f87	Running	t3.micro	Initializing	us-east-1c	ec2-13-220-181-192.co...	1
☐	mobile	i-06de5002e65cdb439	Running	t3.micro	Initializing	us-east-1c	ec2-3-85-34-247.comp...	3
☐	laptop	i-025ac2b5994d24f34	Running	t3.micro	Initializing	us-east-1c	ec2-54-234-234-119.co...	5

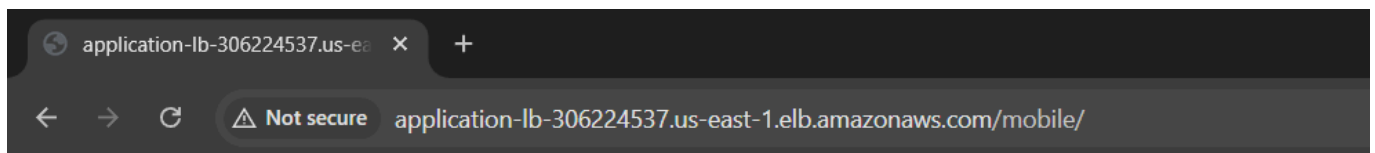
Select an instance

```
---
## Home Page Output
```



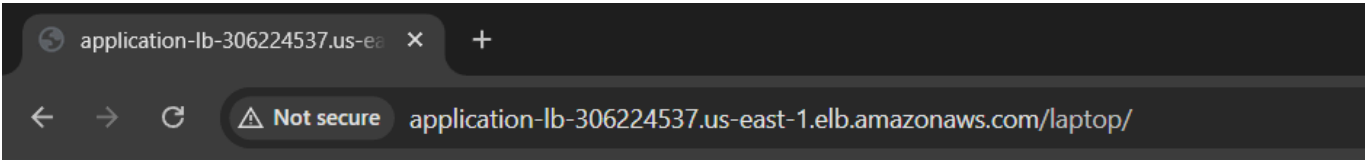
This is Home page ip-172-31-25-121.ec2.internal

Mobile Page Output



This is mobile page ip-172-31-18-63.ec2.internal

Laptop Page Output



This is laptop page ip-172-31-25-162.ec2.internal

Load Balancer Listener Rules

EC2

Dashboard

AWS Global View

Events

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

EC2 > Load balancers > Application-lb > HTTP:80 listener

Protocol:Port
HTTP:80

Load balancer
Application-lb

Default actions

- Forward to target group home-tg: 1 (100%)
Target group stickiness: Off

Listener ARN
arn:aws:elasticloadbalancing:us-east-1:585384907939:listener/app/Application-lb/f5b9d48e944982b4/3bf96a459dd086cb

Rules

Attributes

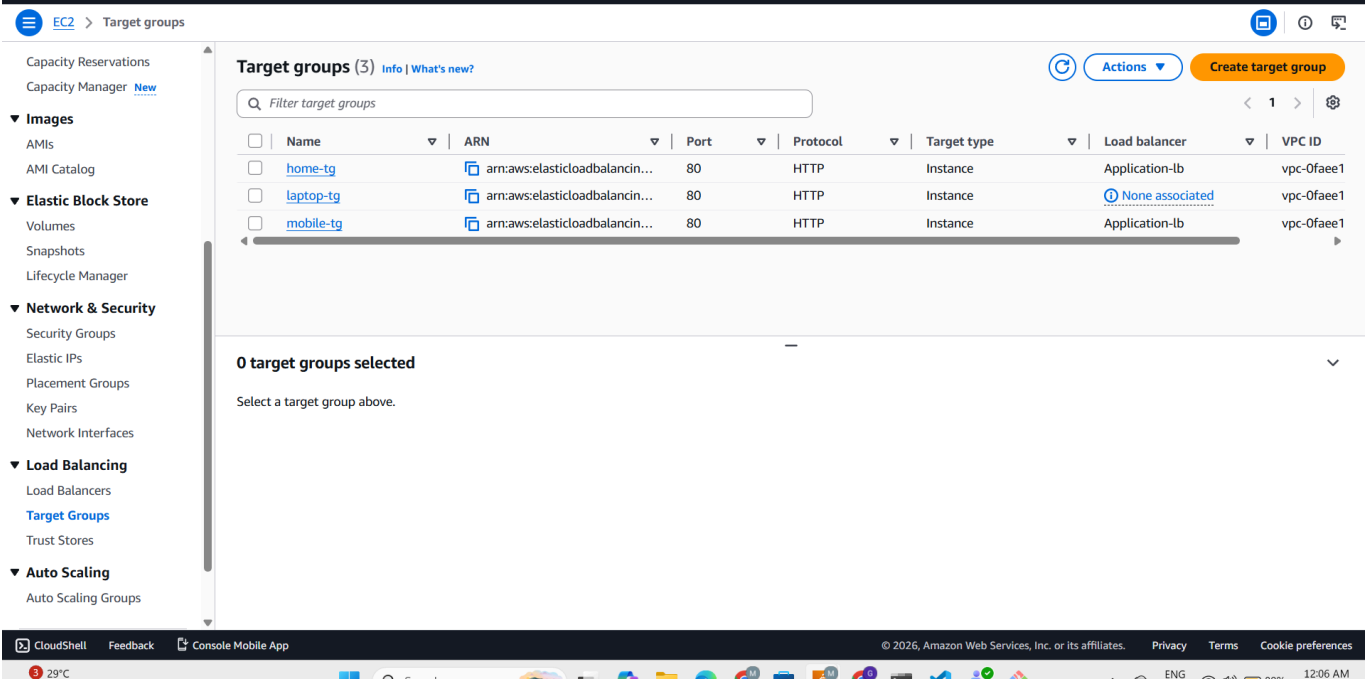
Tags

Listener rules (3)

Traffic received by the listener is routed according to the default action and any additional rules. Rules are evaluated in priority order from the lowest value to the highest value.

	Priority	Name tag	Conditions (If)	Transforms	Actions (Then)	Actions
☐	1	-	Path (value) = /mobile/*	-	Forward to target group mobile-tg	
☐	2	-	Path (value) = /mobile/*	-	Forward to target group mobile-tg	
☐	Last (default)	Default	If no other rule applies	-	Forward to target group home-tg	

Target Groups



Security Group Configuration

Type	Port
SSH	22
HTTP	80

Features

- Application Load Balancer
- Path-Based Routing
- Multiple EC2 Instances
- Round Robin Traffic Distribution
- High Availability
- Load Distribution
- Scalable Architecture
- Centralized Access

Future Improvements

- HTTPS SSL Configuration
- Auto Scaling Group
- Route53 Domain Setup
- Jenkins CI/CD Integration
- Docker Deployment

- Monitoring using CloudWatch
-

Author

Nilesh Pradeep Patil

Fortune Cloud Technology AWS & DevOps Engineering

Summary

This project successfully demonstrates implementation of an AWS Application Load Balancer with multiple EC2 instances and target groups. Separate target groups were created for Home, Mobile, and Laptop services, with two EC2 servers attached to each group for high availability and scalability. The Application Load Balancer distributed traffic using path-based routing and the Round Robin technique, providing efficient traffic management and balanced server utilization in a cloud environment.
